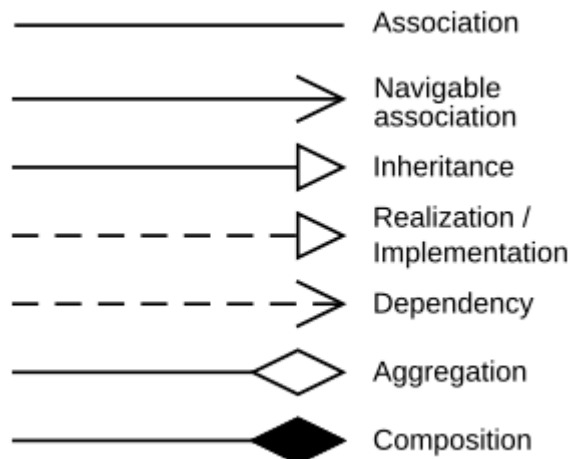


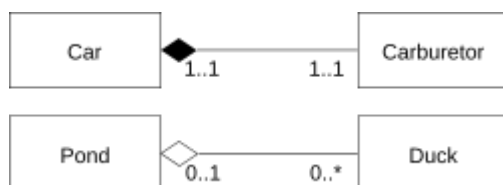
UML

Relaciones



- Asociación: Indica que una clase conoce o utiliza a otra, sin necesariamente contenerla o poseerla.
- Agregación: Es una variante de la relación de asociación que expresa que una clase “tiene un” elemento, representado por otra clase. Ej: Profesor tiene una clase de alumnos. La clase contenida no se destruye junto a la clase contenedora.
- Composición: Indica que una clase es parte de un todo, donde las partes no pueden existir sin el todo. La clase contenida se destruye junto a la clase contenedora.

Multiplicidad



Refactoring

Bad Smells

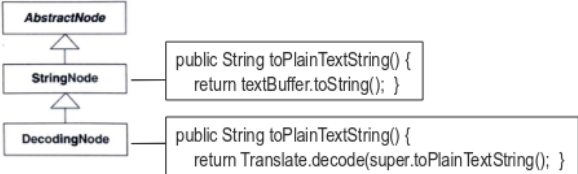
Nombre	Descripcion	Refactorings
Mysterious Name	El nombre de un módulo, variable o clase no comunica claramente que hace y cómo debe usarse.	Rename Method Rename Variable Rename Field
Duplicated Code	La misma estructura de código se encuentra en más de un lugar, unificarlas mejora la legibilidad y la facilidad de realizar cambios.	Extract Method Pull Up Method
Long Method	Un método se extiende demasiado, empeorando su entendimiento.	Extract Method
Large Class	Una clase con muchas variables de instancia o mucho código es propensa al caos y el código repetido.	Extract Class Extract Subclass
Long Parameter List	Un método recibe una larga lista de parámetros.	Replace Parameter with Query Preserve Whole Object Introduce Parameter Object
Divergent Change	Un módulo se modifica frecuentemente de maneras distintas y por distintas razones.	Extract Class
Shotgun Surgery	Al hacer un cambio, también se tienen que realizar pequeñas modificaciones a varias clases distintas.	Move Method Move Field Inline Class
Feature Envy	Un método parece estar más interesado en otra clase que en la suya.	Move Method Extract Method + Move Method
Data Clumps	Cuando tres o cuatro datos aparecen juntos en varios lugares: campos de clases, parámetros en métodos, etc.	Extract Class Introduce Parameter Object
Primitive Obsession	Tipos de datos primitivos se usan en exceso para representar conceptos más complejos, empeorando la mantenibilidad y legibilidad.	Replace Data Value with Object
Switch Statements	La misma estructura de selección múltiple aparece varias veces en distintos lugares del	Replace Type Code with Subclasses Replace Conditional with Polymorphism

	programa, generando código duplicado.	
Lazy Element	Una clase o método, por distintos motivos, ya no es necesario.	Inline Method Inline Class Collapse Hierarchy
Speculative Generality	Hay código o estructuras, con el objetivo de soportar funciones futuras, pero que actualmente son innecesarias.	Collapse Hierarchy Inline Class Remove Parameter
Temporary Field	Una clase que tiene variables de instancia que solo se usan en ciertas circunstancias.	Extract Class Introduce Null Object
Message Chains	Un cliente le pide a un objeto, otro objeto, al cual el cliente le pide otro objeto y así sucesivamente.	Hide Delegate Extract Method + Move Method
Middle Man	La mayoría de los métodos de una clase delegan a otra clase.	Remove Middle Man Inline Method
Inappropriate Intimacy	Clases que se vuelven demasiado íntimas y dedican mucho tiempo en los métodos y variables internas de la otra.	Move Method Move Field Extract Class
Alternative Classes with Different Interfaces	Dos clases realizan tareas idénticas pero tienen interfaces distintas, haciendo difícil la reutilización y llevando a repetir código.	Change Function Declaration Move Method Extract Superclass
Incomplete Library Class	No se puede o es difícil modificar una biblioteca para que haga algo que se necesita y en cambio se hace por fuera de esta.	Introduce Foreign Method Introduce Local Extension
Data Class	Una clase que sólo contiene variables de instancia y métodos para acceder a estas.	Encapsulate Field Encapsulate Collection Extract Method + Move Method
Refused Bequest	Una subclase usa solo algunos de los métodos y propiedades heredados de sus padres.	Push Down Method Push Down Field Replace Inheritance with Delegation
Comments	Se usan comentarios para enmascarar código poco legible o entendible.	Extract Method Rename Method

Refactorings

Nombre	Descripcion	Pasos
Extract Superclass	Existen dos clases con campos y métodos similares, generalmente hay código duplicado.	• Crear superclase abstracta. • Usar Pull Up Field y Method para subir funcionalidad. • Reemplazar el uso de las subclases por la superclase.
Encapsulate Field	Una clase tiene variables de instancia públicas, rompiendo el encapsulamiento.	• Crear setters y getters para el campo. • Reemplazar las menciones del campo con los métodos creados. • Hacer privado el campo.
Move Method	Un método hace más referencia a otro objeto que en el que se encuentra.	• Examinar las funcionalidades usadas por el método, analizar si también deberían moverse. • Declarar el nuevo método en la clase recipiente. • Copiar el código en el nuevo método y ajustarlo para que funcione. • Reemplazar las menciones al viejo método por el nuevo, y luego borrarlo.
Inline Class	Una clase hace casi nada, no es responsable de nada, ni hay planificadas nuevas responsabilidades.	• Crear los campos y métodos a mover en la clase recipiente, los métodos deben delegar a los respectivos de la clase donadora. • Reemplazar todas las referencias a la clase donadora. • Usar Move Method y Move Field para mover los contenidos de la clase donadora hasta que quede vacía. • Borrar la clase donadora.
Replace Loop with Pipeline	El uso de pipelines en reemplazo de los bucles permite mejorar la legibilidad.	• Crear una nueva variable para la colección del loop. • Tomar cada comportamiento en el loop y reemplazarlo con una operación del pipeline. • Remover el loop.
Hide Delegate	Un cliente obtiene un objeto B de un campo o método del objeto A, luego el cliente llama a un método del objeto B, atenta contra el encapsulamiento.	• Crear un método en la clase servidor (A) por cada método de la clase delegado (B) a la que llama el cliente. • Modificar el código para que el cliente use los métodos de la clase servidor (A). • Si el cliente ya no necesita a la clase delegado (A), remover los métodos en la clase servidor (B) que el cliente usaba para acceder a la clase delegado (A)
Replace Conditional with Polymorphism	Hay un condicional que realiza varias acciones dependiendo de las propiedades o el tipo de un objeto.	• Crear la jerarquía que contendrá los comportamientos alternativos. • Si el condicional se encuentra en un método que hace otras cosas, extraerlo con Extract Method. • En cada subclase, redefinir el método que contiene el condicional y copiar el código de la rama correspondiente al método de la subclase. Luego borrar dicha rama. • Una vez que el condicional esté vacío, declarar el método

		como abstracto.
Extract Method	Un fragmento de código en un método puede agruparse junto en un nuevo método, mejorando la cohesión.	• Crear un nuevo método y darle un nombre descriptivo. • Copiar el código relevante al nuevo método, eliminarlo en el método viejo y reemplazarlo por una llamada al nuevo. • Pasar los argumentos necesarios al nuevo método. • De ser necesario, hacer que el nuevo método retorna valores que modifique, y use el otro método.
Replace Temp with Query	Se guarda el resultado de una expresión en una variable local para uso posterior. Al mover la expresión a un método aparte, se mejora la cohesión y la reutilización.	• Asegurar que se hace una sola asignación a la variable. • Usar Extract Method para mover la expresión que se le asigna a la variable, a un nuevo método. • Reemplazar la variable con una consulta con el nuevo método.
Form Template Method	Hay dos métodos en subclases que realizan pasos similares y en el mismo orden, pero son diferentes.	• Descomponer los métodos para que los métodos extraídos sean idénticos o completamente distintos. • Pull Up Method con los métodos idénticos. • Rename Method con métodos distintos para que los métodos de todos los pasos tengan la misma firma. • Pull Up Method de uno de los métodos originales. • Remover los métodos innecesarios.
Replace Conditional Logic with Strategy Method	Existe lógica condicional en un método que controla qué variante ejecutar entre distintas posibles.	• Crear una clase Strategy • Aplicar Move Method para mover el cálculo con los condicionales del contexto al Strategy. • Crear un campo de contexto para el Strategy. • Hacer que el método de contexto delegue a Strategy. • Aplicar Replace Conditional with Polymorphism en el metodo del Strategy.
Replace State-Altering Conditionals with State	Los condicionales que controlan las transiciones de estado de un objeto son complejos.	• Replace Type Code with Class con el campo de estado del contexto para crear la superclase del State. • Extract Subclass para crear un ConcreteState por cada unos de los estados de la clase contexto. • Move Method de contexto a State para cada método que modifique el estado. • Por cada ConcreteState, Push Down Method para mover desde State los métodos que producen una transición desde ese estado, dejar los métodos como abstractos en State.
Move Embellishment to Decorator	Se agregan responsabilidades adicionales a las principales de una clase. Se complica una clase con	• Identificar la superclase o interfaz del objeto a decorar, crearla si no existe. • Aplicar Replace Conditional Logic with Polymorphism,

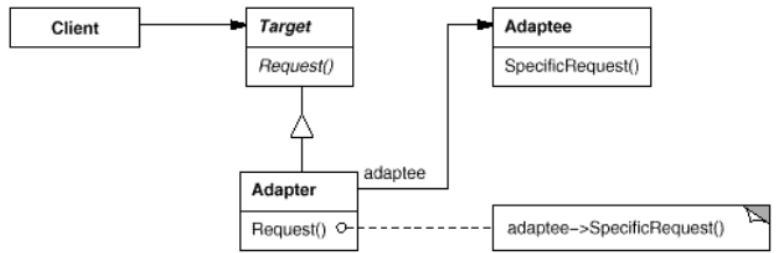
	nuevas variables, métodos y lógica, para cubrir el comportamiento que se ejecuta en casos especiales.	creando al decorator como subclase del decorado.  <pre>classDiagram class AbstractNode { <<abstract>> toPlainTextString() String } class StringNode { toPlainTextString() String } class DecodingNode { toPlainTextString() String } AbstractNode < -- StringNode AbstractNode < -- DecodingNode StringNode --> StringNode : public String toPlainTextString() { return textBuffer.toString(); } DecodingNode --> StringNode : public String toPlainTextString() { return Translate.decode(super.toPlainTextString()); }</pre>
--	---	---

Patrones

Adapter

Propósito: Convierte la interfaz de una clase en otra interfaz que es la que esperan los clientes. Permite que cooperen clases que de otra forma no podrían por tener interfaces incompatibles.

Participantes:



- Target: Define la interfaz específica del dominio que usa el cliente.
- Adaptee: Define una interfaz existente que necesita ser adaptada.
- Adapter: Adapta la interfaz de Adaptee a la interfaz Target.

Desventajas: Aumenta la complejidad del código al introducir nuevas interfaces y clases.

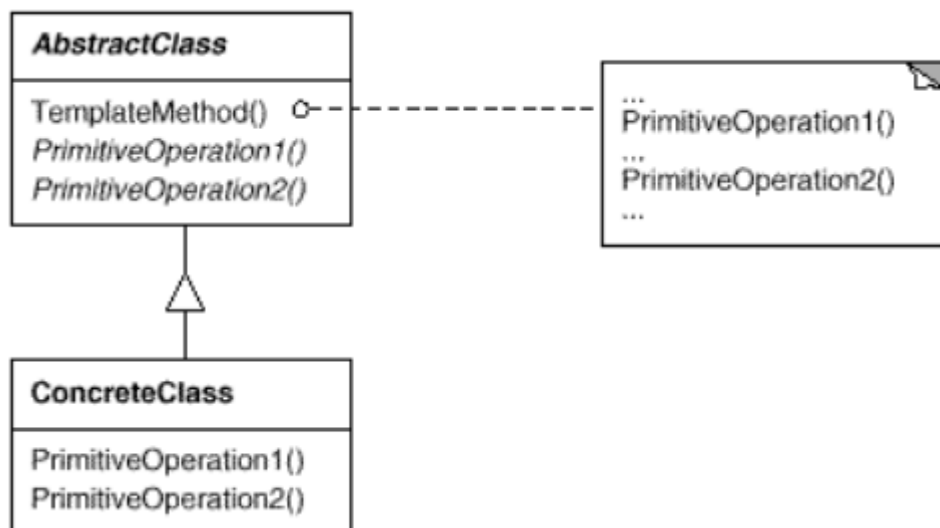
Colaboraciones: Los clientes llaman a operaciones de una instancia de Adapter. A su vez el Adapter llama a operaciones de Adaptee, que son las que satisfacen la petición.

NOTA: El ejemplo mostrado corresponde al adaptador de objetos, que se basa en composición. Si mediante la herencia múltiple, Adapter hereda SpecificRequest() de Adaptee, hablamos de un adaptador de clases.

Template Method

Propósito: Define en una operación el esqueleto de un algoritmo, delegando en las subclases algunos de sus pasos. Permite que las subclases redefinan ciertos pasos de un algoritmo sin cambiar su estructura.

Participantes:



- **AbstractClass**
 - Define operaciones primitivas abstractas que son definidas por las subclases para implementar los pasos de un algoritmo.
 - Implementa el template method que define el esqueleto de un algoritmo. El template method llama a las operaciones primitivas así como a operaciones definidas en Abstract Class o en otros objetos.

- ConcreteClass: Implementa las operaciones primitivas para realizar los pasos del algoritmo específicos en las subclases.

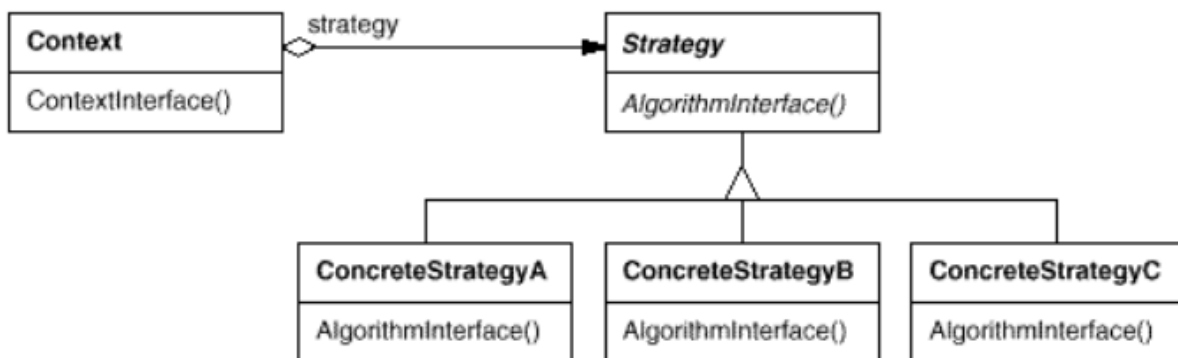
Desventajas: Algún cliente puede ser limitado por el esqueleto del algoritmo provisto.

Colaboraciones: ConcreteClass se basa en AbstractClass para implementar los pasos de un algoritmo que no cambian.

Strategy

Propósito: Define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables. Permite que un algoritmo varíe independientemente de los clientes que lo usan.

Participantes:



- Context:
 - Se configura con un objeto ConcreteStrategy.
 - Mantiene una referencia a un objeto Strategy.
 - Puede definir una interfaz que le permita al Strategy acceder a sus datos.
- Strategy: Declara una interfaz común a todos los algoritmos permitidos. Context usa esta interfaz para llamar al algoritmo definido por un ConcreteStrategy.
- ConcreteStrategy: Implementa el algoritmo usando la interfaz Strategy.

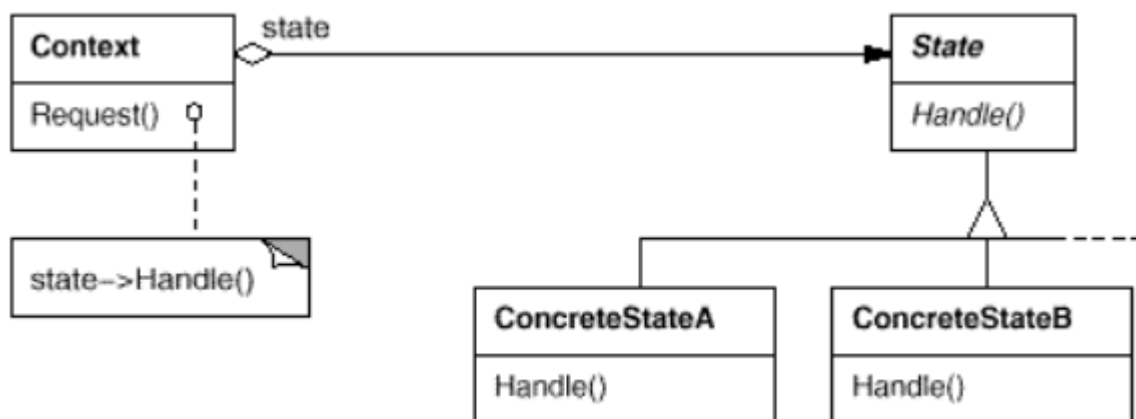
Desventajas: Si solo existen un par de algoritmos y raramente cambian, puede que no sea buena idea complicar el programa con nuevas clases e interfaces.

Colaboraciones: Un contexto redirige peticiones de los clientes a su estrategia. Los clientes crean un objeto ConcreteStrategy, el cual pasan al contexto; por tanto, los clientes interactúan exclusivamente con el contexto.

State

Propósito: Permite que un objeto modifique su comportamiento cada vez que cambia su estado interno. Parecerá que cambia la clase del objeto.

Participantes:



- Context:
 - Define la interfaz de interés para los clientes.
 - Mantiene una instancia de una subclase ConcreteState que define el estado actual.
- State: Define una interfaz para encapsular el comportamiento asociado con un determinado estado de Context.
- Subclases ConcreteState: Cada subclase implementa un comportamiento asociado con un estado de Context.

Desventajas: Puede ser demasiado si solo existen pocos estados o raramente cambian.

Colaboraciones:

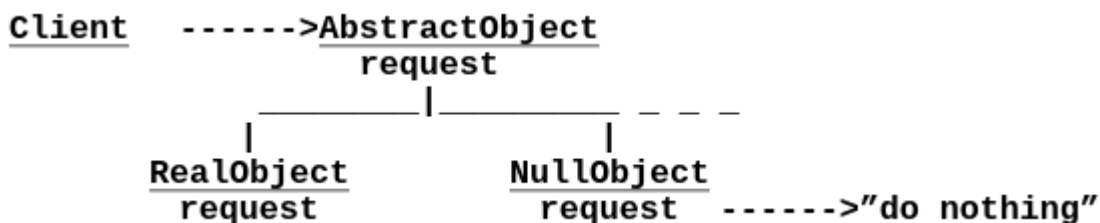
- Context delega las peticiones que dependen del estado en el objeto ConcreteState actual.
- Context puede pasarse a sí mismo como parámetro para que State maneje la petición. Esto permite a State acceder al contexto si fuese necesario.

- Context es la interfaz principal para los clientes. Los clientes pueden configurar un Context con un State y luego no necesitan tratar con objetos State directamente.
- Context o ConcreteState pueden determinar qué estado sigue a otro y bajo qué circunstancias.

Null Object

Propósito: Proporciona un sustituto para otro objeto que comparte la misma interfaz pero no hace nada. El Null Object encapsula las decisiones de implementación sobre cómo "no hacer nada" y oculta esos detalles a sus colaboradores.

Participantes:



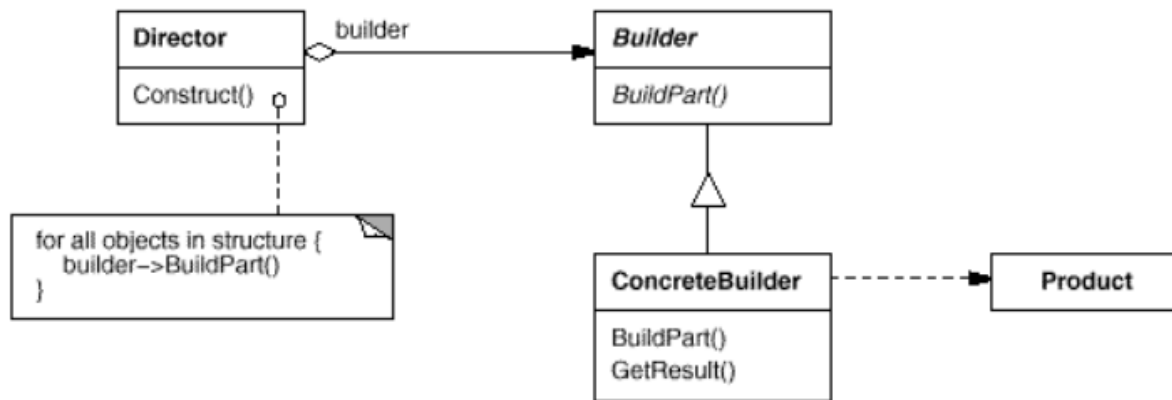
- Client: Usa objetos AbstractObject.
- AbstractObject: Define la interfaz común para todos los objetos reales y el Null Object.
- RealObject: Implementa el comportamiento "normal" o "útil" definido por la interfaz.
- NullObject: Implementa la misma interfaz que el RealObject pero no hace nada.

Desventajas: Puede ser difícil de implementar si varios clientes no acuerdan como el Null Object debería "hacer nada".

Builder

Propósito: Separa la construcción de un objeto complejo de su representación, de forma que el mismo proceso de construcción pueda crear diferentes representaciones (Ej. Casa de madera, de piedra, etc).

Participantes:



- **Builder**: Especifica una interfaz abstracta para crear las partes de un objeto Product.
- **ConcreteBuilder**:
 - Implementa la interfaz Builder para construir y ensamblar las partes del producto.
 - Define la representación a crear.
 - Proporciona una interfaz para devolver el producto.
- **Director**: Construye un objeto usando la interfaz Builder.
- **Product**:
 - Representa el objeto complejo en construcción.
 - Incluye las clases que definen sus partes constituyentes, y las interfaces para ensamblar dichas partes en el resultado final.

Desventajas: La complejidad general del código aumenta, ya que el patrón requiere la creación de múltiples clases nuevas.

Colaboraciones:

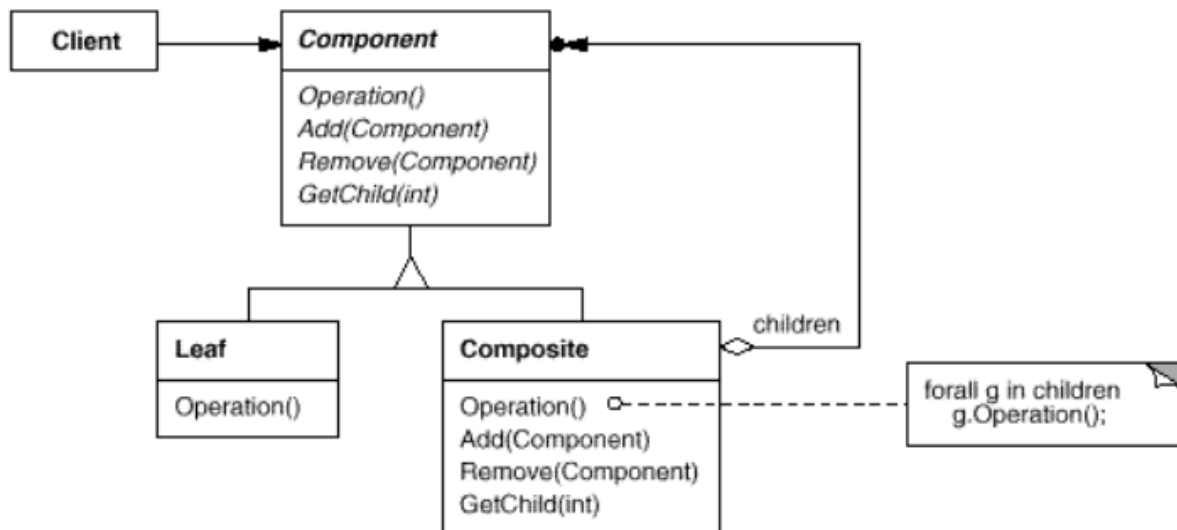
- El cliente crea el objeto Director y lo configura con el Builder deseado.
- El Director notifica al Builder cada vez que hay que construir una parte del producto.
- El Builder maneja las peticiones del director y las añade al producto.
- El cliente obtiene el Product del Builder.

NOTA: Si varios ConcreteBuilder realizan la misma parte o paso de la misma manera, su implementación se puede especificar en Builder, para que estos la hereden. Si al hacer esto existe otro ConcreteBuilder que no la necesita, puede redefinirla.

Composite

Propósito: Compone objetos en estructuras de árbol para representar jerarquías de parte-todo. Permite que los clientes traten de manera uniforme a los objetos individuales y a los compuestos.

Participantes:



- **Component:** Declara la interfaz de los objetos de la composición.
- **Leaf:** Define el comportamiento de los objetos primitivos de la composición, no tiene hijos.
- **Composite:** Define el comportamiento de los componentes que tienen hijos. Tiene hijos. Implementa las operaciones relacionadas con los hijos.
- **Client:** Manipula objetos en la composición a través de la interfaz **Component**.

Desventajas: Puede ser difícil proveer una interfaz común para clases cuya funcionalidad difiere mucho.

Colaboraciones: Los clientes usan la interfaz de la clase **Componente** para interactuar con los objetos de la estructura compuesta. Si el recipiente es una hoja, la petición se trata correctamente. Si es un compuesto, normalmente redirige las peticiones a sus componentes hijos, posiblemente realizando operaciones adicionales antes o después.

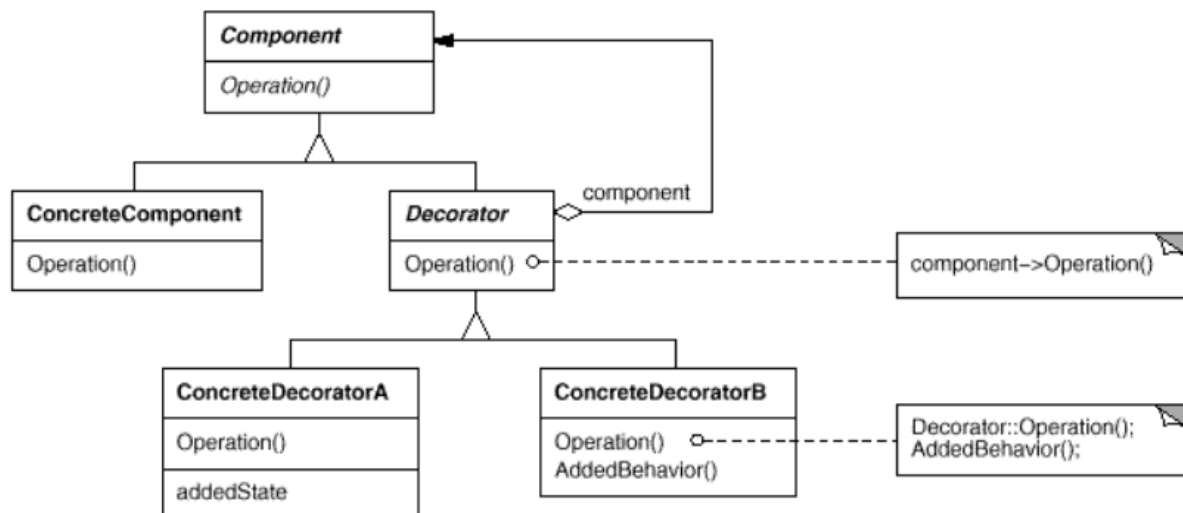
NOTA: Existen dos variantes a la hora de implementar este patrón, el mostrado corresponde al **diseño para la transparencia** que permite que

los clientes traten a Leaf y Composite de la misma manera. Pero se pierde la seguridad de tipos porque los clientes pueden ejecutar operaciones relacionadas a hijos sobre una hoja. La otra variante corresponde al **diseño para la seguridad de tipos** en la cual Component solo define Operation() y las operaciones sobre los hijos se definen en Composite.

Decorator

Propósito: Asigna responsabilidades adicionales a un objeto dinámicamente, proporcionando una alternativa flexible a la herencia para extender la funcionalidad.

Participantes:



- **Component:** Define la interfaz para objetos a los que se puede añadir responsabilidades dinámicamente.
- **ConcreteComponent:** Define un objeto al que se pueden añadir responsabilidades adicionales.
- **Decorator:** Mantiene una referencia a un objeto **Component** y define una interfaz que se ajusta a la interfaz de **Component**.
- **ConcreteDecorator:** Añade responsabilidades al componente.

Desventajas:

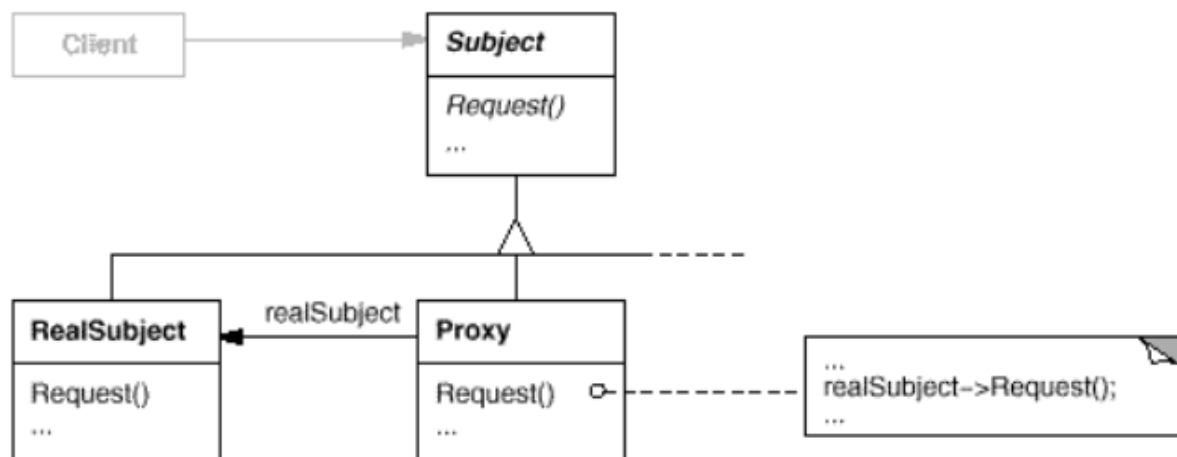
- Es difícil remover un decorador específico de un stack.
- Es difícil implementar un decorador de manera que su comportamiento no dependa del orden de los decoradores en el stack.

Colaboraciones: El decorador redirige peticiones a su objeto componente. Opcionalmente puede realizar operaciones adicionales antes y después de reenviar la petición. A partir de un Component, el cliente puede crear un ConcreteDecorator para agregar la responsabilidad decorada, como ConcreteDecorator a su vez es un Component, este proceso puede repetirse, resultando en un stack de decoradores.

Proxy

Propósito: Proporciona un representante o sustituto de otro objeto para controlar el acceso a este.

Participantes:



- Proxy:
 - Mantiene una referencia que le permite acceder al objeto real.
 - Provee una interfaz idéntica a la de Subject para que pueda sustituir a RealSubject.
 - Controla el acceso al sujeto real, y puede ser responsable de su creación y borrado.
 - Según su tipo pueden tener otras responsabilidades:
 - **Proxy remoto:** Codifican la petición y sus argumentos para enviarla al sujeto real que está en un espacio de direcciones distinto.
 - **Proxy virtual:** Pueden guardar información adicional sobre el sujeto real para posponer el acceso a este.
 - **Proxy de protección:** Comprueban que el llamador tenga los permisos de acceso necesarios para realizar una petición.

- Subject: Define la interfaz común para RealSubject y Proxy, de modo que pueda usarse Proxy en cualquier sitio en el que se espere RealSubject.
- RealSubject: Define el objeto real representado.

Desventajas: Podría retrasar la respuesta del servicio.

Colaboraciones: Proxy redirige peticiones a RealSubject cuando sea necesario, dependiendo del tipo de proxy.